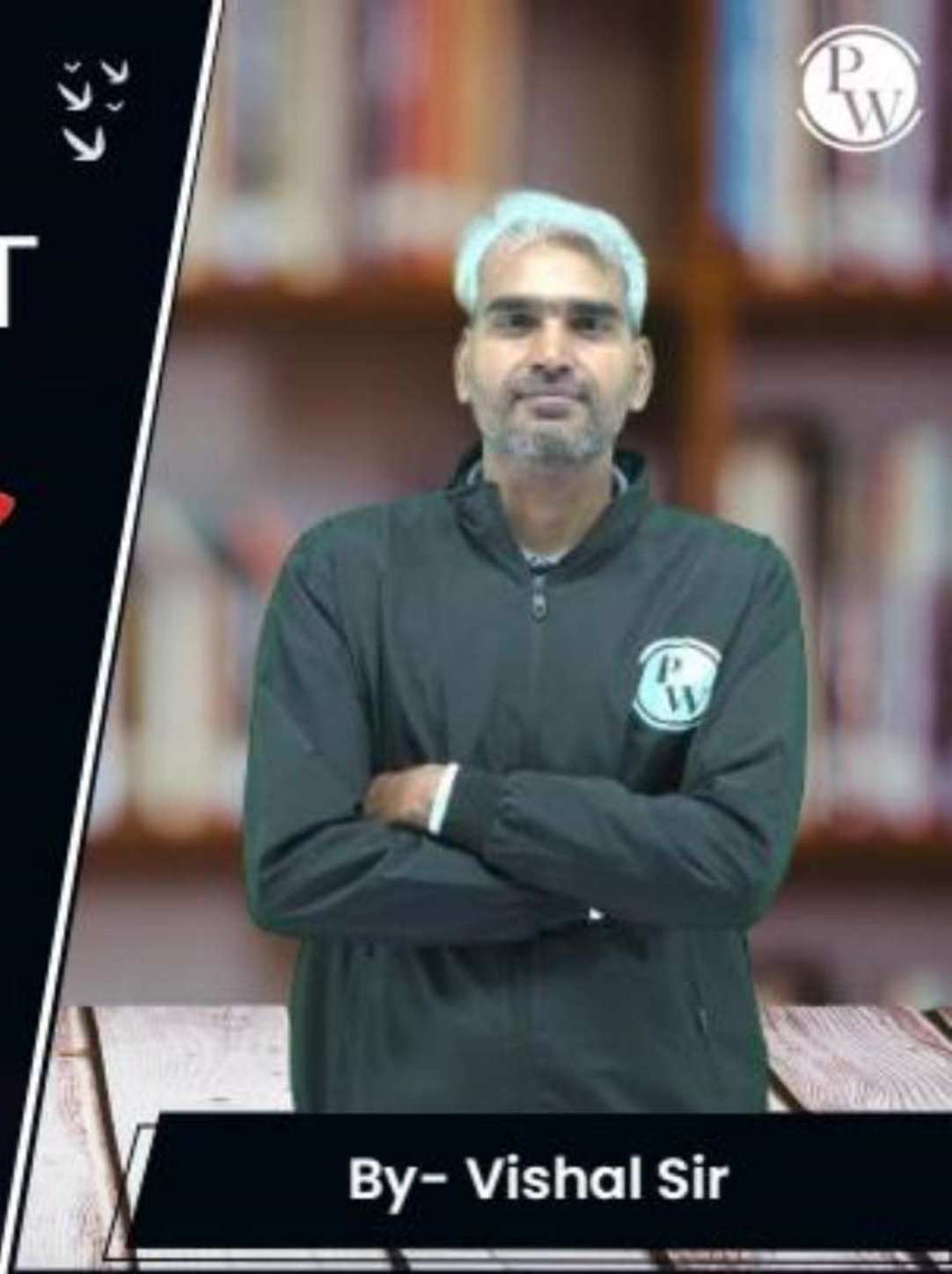
Computer Science & IT



Compiler Design

Introduction and Lexical Analysis

Lecture No. 02



By- Vishal Sir

Recap of Previous Lecture



- Topic: Introduction
- Topic: Syllabus and Weightage
- Topic: Language Processing System
- Topic: Phases of Compiler

Slide

Topics to be Covered



Topic Quick Revision

Topic Phases of Compiler

Topic Translators

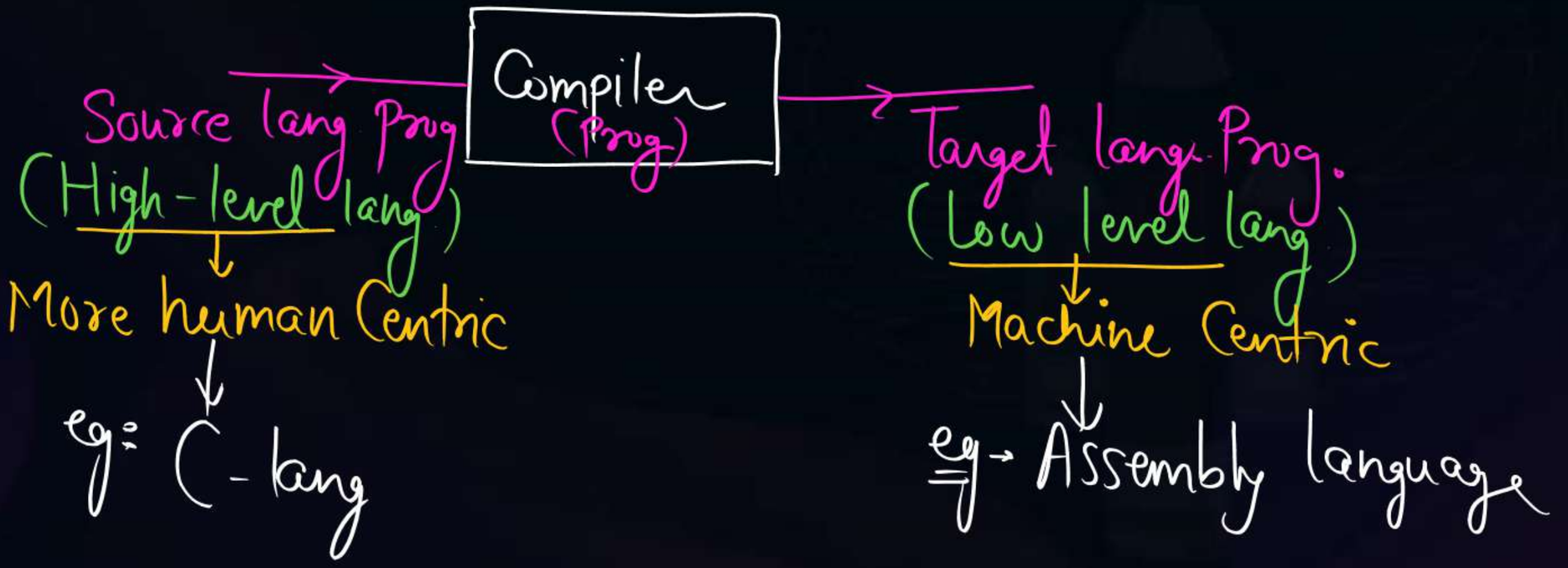
Slide



Topic : Quick Revision

Introduction to Compiler

Compiler :- Compiler is a program that translates Source language prog. into target lang. prog

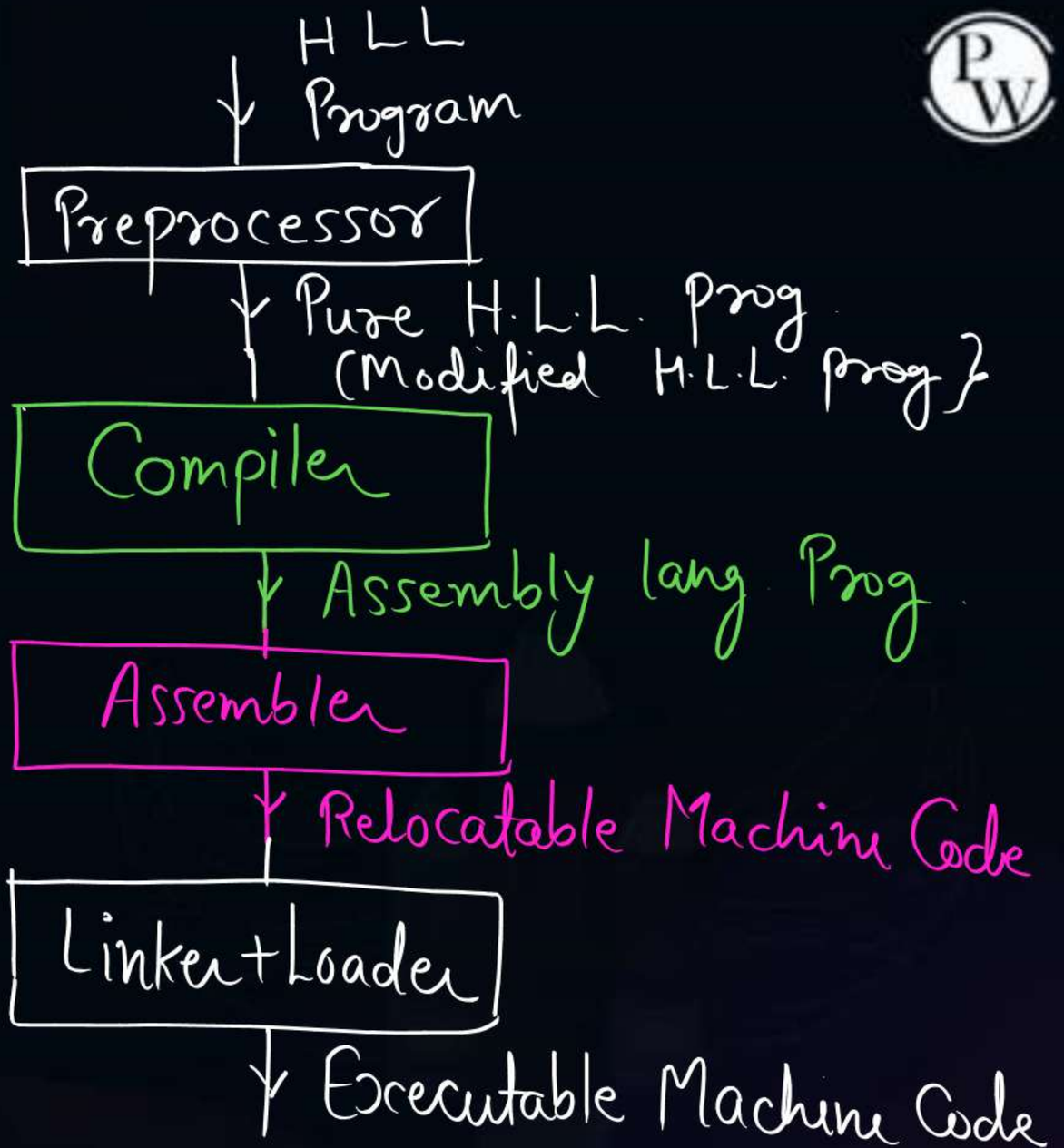


Slide



Topic : Quick Revision

Language Processing System



Slide

Preprocessor :- Preprocessor is a program that includes all the files specified in the main program, and it also performs the task of macro expension.

main_prog(main.c)

```
#include<ab.c>
# define Size 10
main( )
  { int a[size];
    f( );
  }
```

ab.c

```
f1( )
{

}
```

Slide

* Assembler :- Assembler will take the o/p of compiler (Assembly lang code) and translates that prog into binary {Machine language} but it does not resolve external memory references

Slide

Linker + Loader :-

* Loader is used to load the binary program into the memory
* Linker is used to resolve all external references

Eg:
```
extern int x;
Printf("%d", x);
```



Topic : Phases of Compiler

↓ Stream of characters

Lexical Analysis

↓ Stream of tokens/lexemes

Syntax Analysis

↓ Parse tree (Syntax Tree)

Semantic Analysis

(Decorated Parse Tree) = ↓ Abstract Syntax Tree (or Annotated Parse Tree)

Intermediate Code Generation

↓ Intermediate Representation (Three Code Add)

Machine Independent Optimization

↓ optimized Intermediate Representation

(Assembly) Target Code Generation

↓ Target language Code

Machine dependent Optimization

↓ optimized Target lang. Code

{ Check for meaningful words

{ Check whether the Statement is Syntactically correct or not.

{ Check whether the Statement is Semantically Correct or not
i.e. Meaningful or not

Slide



Topic : Phases of Compiler

(Analysis Phase) → Source program is analyzed

front-end of the Compiler → Source lang dependent & Machine Independent

(Synthesis phase) → Analysis of front-end is used to Synthesize Target Code

Back-end of Compiler → Machine dependent

↓ Stream of characters

Lexical Analysis

↓ Stream of tokens/lexemes

Syntax Analysis

↓ Parse tree (Syntax Tree)

Semantic Analysis

(Decorated Parse Tree) = ↓ Abstract Syntax Tree (or Annotated Parse Tree)

Intermediate Code Generation

↓ Intermediate Representation (Three Add Code)

Machine Independent Optimization

↓ optimized Intermediate Representation

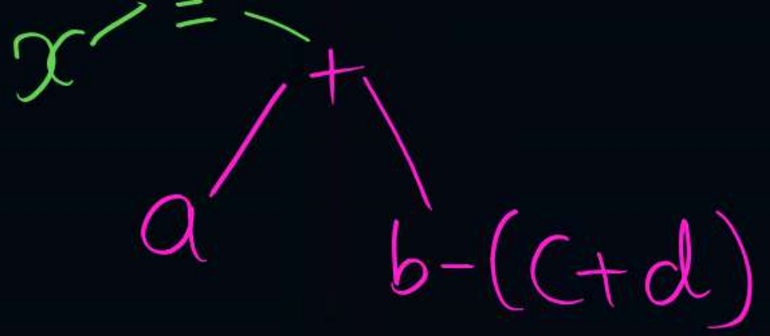
(Assembly) Target Code Generation

↓ Target language Code

Machine dependent Optimization

↓ optimized Target lang. Code

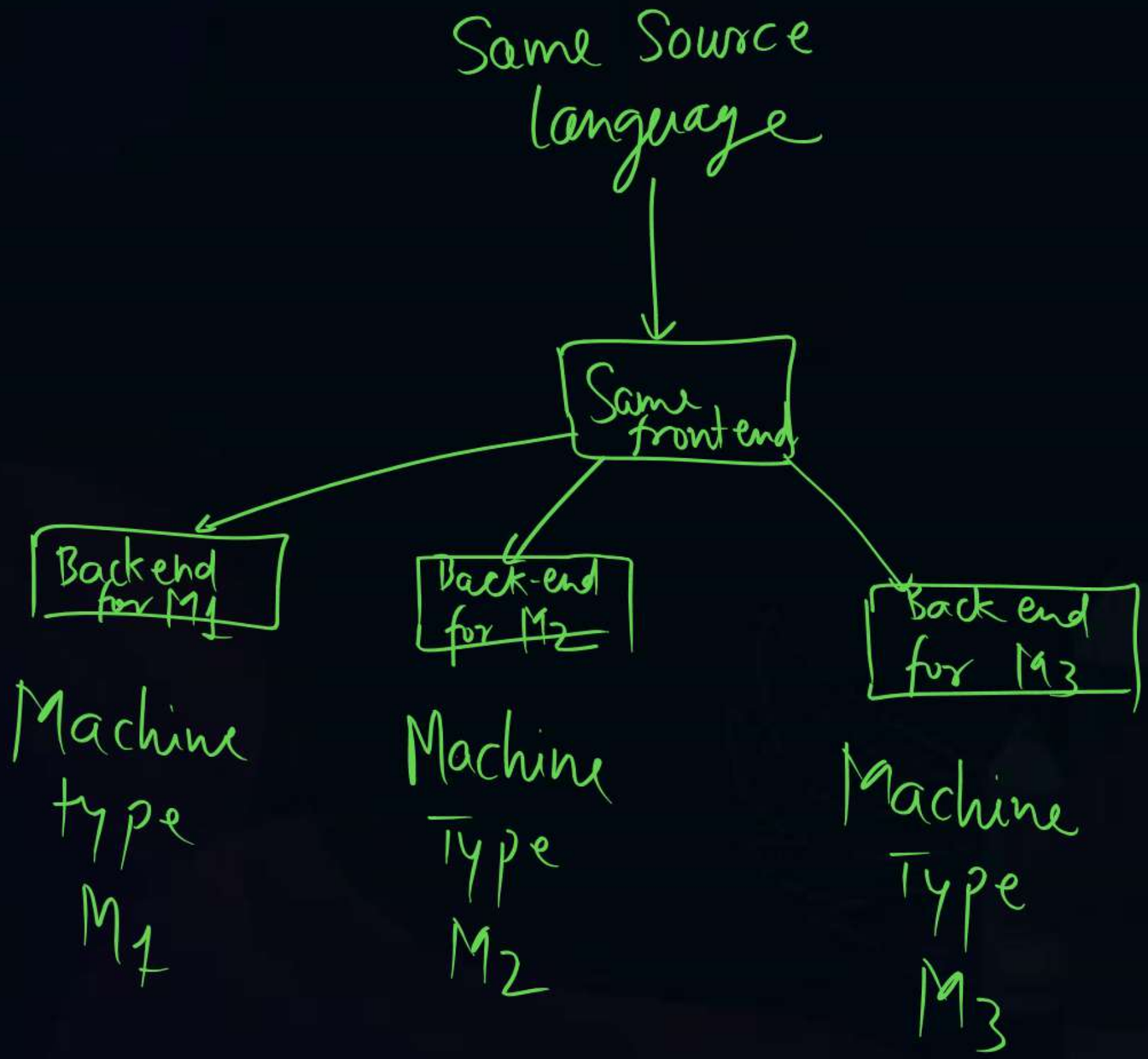
Slide



$$x = a + \underbrace{\underbrace{\underbrace{b - \underbrace{(c+a)}_{t_1}}_{t_2}}_{t_3}}$$

$$\begin{cases} t_1 = c + d \\ t_2 = b - t_1 \\ x = a + t_2 \end{cases}$$

Slide


Same Source language
Same front end
Back end for M1
Back-end for M2
Back end for M3
Machine type M1
Machine Type M2
Machine Type M3

Slide



Topic : Translators

↳ A prog that translates one form of prog into another form

eg of translators :-

1. Compiler ✓
2. Interpreter ✓
3. Assembler
4. Preprocessor

Slide



2 mins Summary

- **Topic** Lexical Analysis
- **Topic** Syntax Analysis
- **Topic** Semantic Analysis
- **Topic** Intermediate Code Generation
- **Topic** Target Code Generation

THANK - YOU